Reversible Oracles Preserving Integer LP-Pruning Decisions for the 0/1 Knapsack Problem

Nagini S.[1], Madhavi A.[1*], Shiva Sai[1], Metta Kumudini[1], Purusharth Mishra[1], Rakshit Chaturvedi[1], Veerabhadra Yerram[1]

[1]Department of Computer Science and Engineering, VNR VJIET, Hyderabad, India.

*Corresponding author(s). E-mail(s): madhavi_a@vnrvjiet.in;

Abstract

Quantum branch-and-bound frameworks use reversible bounding predicates, whose implementation costs can offset reductions in oracle queries. We study the linear-programming relaxation of the 0/1 knapsack problem under a fixed efficiency-ordered prefix branching rule. A classically certified finite bank of rational affine bounds preserves every integer-threshold pruning decision without reproducing the fractional bound itself. We derive coverage certificates, minimum-cardinality constructions on a density grid and over nonnegative rational multipliers, and a sufficient construction that avoids capacity-reachability dynamic programming. A counterexample family shows that a linear number of multipliers can still be necessary. The reversible construction uses constant-coefficient arithmetic, explicit register bounds, and compute–copy–uncompute cleanup. Exact arithmetic checks on 700 original instances and their 700 reduced cores cover 5,204,456 reachable states with zero mismatches in 20,817,824 bank-floor comparisons. Completed small-circuit basis tests report zero gate mismatches in 1,120 Fourier-arithmetic basis columns and 2,216 Boolean gate columns; these tests do not constitute large-instance circuit simulation. On 105 fixed-depth instances, derived logical resources relative to a root-aware fractional-scan reconstruction have median bank-to-control ratios of 0.4981 for controlled-NOT gates, 0.4908 for active qubits, and 1.0720 for depth. The results establish scoped predicate preservation and a gate-count–depth tradeoff, not hardware advantage or end-to-end quantum speedup.

Keywords: quantum computing, knapsack problem, reversible arithmetic, linear programming, branch-and-bound, quantum oracles

1 Introduction

The 0/1 knapsack problem asks for a subset of maximum total value whose total weight does not exceed a given capacity. Linear-programming (LP) bounds and classical core methods exploit structure beyond enumeration [1–3]. Quantum backtracking and branch-and-bound (B&B) frameworks motivate a different implementation question: how much reversible computation is required to decide that a partial assignment cannot reach an integer objective target? Their query guarantees assume access to the required oracles [4, 5]; an oracle query is not a unit of physical runtime.

The distinction between a numerical bound and its pruning predicate is useful here. For integer profits and an integer target, two fractional upper bounds produce identical pruning decisions whenever their floors agree. A reversible oracle can therefore preserve the decision without returning the fractional LP value. The difficulty is to establish this equality at every allowed prefix, rather than selecting a few multipliers that merely give valid but weaker bounds.

We study a static, instance-dependent bank of nonnegative rational multipliers under a fixed nonincreasing exact-efficiency order. Our construction links all-prefix coverage to interval stabbing and compiles the resulting bank into a clean reversible predicate. Affine Lagrangian bounds, minimization over multiplier sets, integer rounding, interval stabbing, and reversible arithmetic are established techniques [6–9]. The contribution is a construction combining these techniques with coverage certificates for the integer LP predicate and a matched reversible-resource comparison.

The paper makes four concrete contributions. First, it specifies the exact prefix and integer-threshold contract and derives finite-grid and rational-bank coverage certificates. Second, it gives a sufficient construction without capacity-reachability dynamic programming and a family requiring linearly many multipliers. Third, it describes a reversible constant-coefficient implementation with bounded unsigned intermediates

and cleaned workspace. Fourth, it evaluates arithmetic coverage, small-circuit behavior, and logical resources separately. The comparison with a root-aware scan control shows lower gate counts and qubit requirements alongside increased median depth.

Sections 3–4 develop the predicate and coverage results. Section 5 describes the circuit and comparison control. Sections 6 and 7 give the evaluation protocol and findings, and Section 8 discusses their scope.

2 Literature Review

Classical exact methods. Fractional LP bounds have a long history in discrete optimization [1]. The Horowitz–Sahni meet-in-the-middle (MitM) algorithm splits the items into two halves and combines partial solutions, providing a classical $O(2^{n/2})$ arithmetic-operation baseline [10]. Kellerer, Pferschy, and Pisinger consolidate dynamic programming, greedy bounds, and branch-and-bound (B&B) [2]. The *core* concept was introduced by Balas and Zemel [3]; Pisinger subsequently showed why core difficulty must be assessed across capacities and instance distributions [11]. MINKNAP uses an expanding-core dynamic program [12], while COMBO combines dynamic programming with strong bounds [13]. These methods provide stronger classical context than brute-force baselines alone. Beier and Vöcking's expected-polynomial knapsack analysis [14] and Dey, Dubey, and Molinaro's random binary-IP B&B analysis [15] require their respective distributional assumptions; neither establishes polynomial search trees for the benchmark classes considered here.

Quantum search and its limits. For an unstructured space of size N containing one marked item, Grover's algorithm finds that item with bounded error using $O(\sqrt{N})$ oracle queries [16]. Amplitude amplification generalizes this principle [17], with optimization variants via pure adaptive search [18] and Dürr–Høyer minimum finding [19]. Treating the 2^n binary assignments as an unstructured objective table gives an $O(2^{n/2})$-query minimum-finding baseline at constant success probability, before

accounting for oracle cost [19]. Bennett et al.'s black-box lower bounds [20] concern unstructured oracle access; they do not establish an unrestricted lower bound for structured knapsack algorithms or equate query complexity with runtime.

Subset-sum algorithms and memory assumptions. Closely related subset-sum work includes the representation technique of Becker, Coron, and Joux [21] and the quantum algorithms of Bernstein et al. [22] and Helm–May [23]. Their input models and, where used, heuristic assumptions must be distinguished from worst-case optimization with separate weights and values. Bonnetain et al. identify a problem with a walk-update heuristic and develop alternative algorithms and memory tradeoffs [24]; this is not a rejection of every quantum subset-sum algorithm. Jaques and Rattew distinguish quantum random-access memory (QRAM) architectures and their resource assumptions [25]. Such costs require explicit accounting, rather than a blanket assertion that quantum-addressable memory is impossible.

Gate-model knapsack and hybrid optimization. Hou and Perkowski give gate-model bounded-knapsack circuits that test complete candidate assignments [26]. Hamitouche et al. combine Grover-based feasibility and threshold tests with a separately computed classical upper bound, for which a fractional-knapsack bound is one option [27]. Wilkening et al. present a quantum tree generator (QTG) with amplitude amplification; separately, their quantum B&B (QBnB) benchmarks use fractional-greedy bounds [28]. Their released R1 software and experiment data provide a version-specific implementation reference [29]. These works make quantum knapsack arithmetic and the use of fractional bounds prior art. Christiansen et al. combine feasible-solution state preparation with the Grover-mixer quantum approximate optimization algorithm (QAOA) [30], within the variational framework introduced by Farhi et al. [31]. Christiansen et al. establish arbitrarily high optimal-solution sampling probability at

sufficiently large depth with suitable parameters [30]; this does not establish a practical finite-depth advantage. Grover adaptive search for constrained polynomial binary optimization [32] is a distinct search approach, rather than a variational algorithm.

LP and Lagrangian bounds in quantum methods. Sanavio et al. combine classical B&B with quantum annealing for integer linear problems [33]. Bożejko et al. use Lagrangian multiplier bounds in a D-Wave knapsack B&B framework, including minimization over a multiplier set and integer rounding in pruning [6]. Their large-instance device experiments evaluate bounds at the root, rather than executing the full search tree on the quantum processing unit (QPU). Gabbassov et al. study Lagrangian duality for constrained quantum optimization and compare it with quadratic unconstrained binary optimization (QUBO) reformulations [34]. Thus, neither Lagrangian relaxation nor taking an integer floor of a valid upper bound is itself a new contribution.

Query frameworks and reversible implementation. Montanaro's quantum backtracking [4] and quantum B&B [5] give query reductions under specified access to tree and bounding oracles. These frameworks do not make the cost of implementing an LP oracle free. Martiel and Remaud study concrete circuit implementation of quantum backtracking [35]. Reversible arithmetic includes the networks of Vedral, Barenco, and Ekert [8], Draper's Fourier-basis addition [9], and Cuccaro et al.'s ripple-carry addition [36]; elementary controlled-gate decompositions are treated by Barenco et al. [37]. Thapliyal et al.'s reversible integer divider [38] is a useful arithmetic comparison, but the constant-coefficient oracle construction considered here does not execute that divider. Minimum interval stabbing is also an established combinatorial problem [7], so any contribution using it must concern the reduction and its endpoint conditions, rather than the greedy algorithm itself. Preskill's discussion of noise and depth constraints [39] motivates separating logical circuit resources from device performance. Qiskit [40] supplies software tools; its use alone is not evidence of execution on a quantum processor.

Table 1 Related methods and the distinctions relevant to an LP-pruning oracle. These descriptions identify scope, not a ranking of performance.

Work	Computation or model	Relevant scope
Montanaro [4, 5]	Quantum backtracking and B&B query frameworks	Assumes access to the required tree and bounding oracles.
Hou–Perkowski [26]; Hamitouche et al. [27]	Gate-model candidate feasibility and objective tests	Complete-assignment tests; Hamitouche et al.'s optional fractional bound is classical.
Wilkening et al. [28, 29]	QTG search; separate fractional-bound QBnB comparison	Published arithmetic primitives and released simulator/resource models; a matched LP-oracle circuit requires reconstruction.
Christiansen et al. [30]	Feasible-solution preparation and QAOA	Variational search with depth-dependent behavior.
Sanavio et al. [33]; Bożejko et al. [6]	Classical B&B with quantum-annealing subproblems	A different computational model from a coherent reversible prefix predicate.
Martiel–Remaud [35]	Circuit implementation of quantum backtracking	Implementation resources remain separate from abstract query counts.

Closest prior work and our position. The relevant distinction is between computing a fractional bound, testing a complete assignment, and reversibly deciding whether the LP relaxation of a partial prefix excludes an integer target. The construction under study targets the last task: for a fixed exact-efficiency prefix order, a classically certified bank of affine bounds is compiled into a reversible predicate that preserves the integer-threshold LP-pruning decisions. This scope requires positive integer weights, nonnegative integer profits, a coverage certificate, representable integer thresholds, a separate capacity check, and clean initial workspace. It does not imply reproduction of the numerical fractional LP value, support for arbitrary branching orders, or an implementation of the complete quantum walk. Circuit comparisons reconstructed from published fractional-greedy primitives [28, 29] must therefore account for both predicate semantics and resource costs, and be identified as reconstructions rather than released LP-oracle circuits. Table 1 summarizes the roles of the closest approaches without asserting historical priority or a hardware advantage.

3 Problem and pruning contract

Consider n items with weights $w_i \in \mathbb{Z}_{>0}$, profits $v_i \in \mathbb{Z}_{\geq 0}$, and capacity $W \in \mathbb{Z}_{\geq 0}$. We maximize $\sum_{i=1}^{n} v_i x_i$ subject to $\sum_{i=1}^{n} w_i x_i \leq W$ and $x_i \in \{0, 1\}$. Relabel the items so that $\rho_i = v_i / w_i$ is nonincreasing. Comparisons use exact rational arithmetic; ties retain a fixed deterministic order. Distinct efficiencies are not assumed. The empty instance is allowed.

At depth d, the first d decisions are fixed. Write

$$P = \sum_{i \leq d} v_i x_i, \qquad R = W - \sum_{i \leq d} w_i x_i. \tag{1}$$

For a feasible prefix, $R \geq 0$, the residual LP value is

$$U_d(R) = \max \left\{ \sum_{i > d} v_i y_i : \ \sum_{i > d} w_i y_i \leq R, \ 0 \leq y_i \leq 1 \right\}. \tag{2}$$

The target predicate, with 1 meaning rejection, is

$$f(x, \tau) = \begin{cases} 1, & R < 0, \\ [P + U_d(R) < \tau], & R \geq 0. \end{cases} \tag{3}$$

Here τ is an integer; in a circuit it must be representable in the chosen unsigned register. For an integer incumbent L, set $\tau = L+1$. Rejection then means $\lfloor P + U_d(R) \rfloor \leq L$, which differs from the unrounded test $P + U_d(R) \leq L$. Rejection rules out every feasible completion reaching τ; nonrejection does not guarantee such a completion.

For $\lambda \geq 0$, define the residual dual bound

$$F_d(R, \lambda) = \lambda R + \sum_{i > d} \max(v_i - \lambda w_i, 0). \tag{4}$$

For every feasible LP vector, $v_i y_i \le \lambda w_i y_i + \max(v_i - \lambda w_i, 0)$; summing proves $U_d(R) \le F_d(R, \lambda)$. Greedy fractional filling in efficiency order attains equality at the split item's density: items of higher density are filled, those of lower density are empty, and the split group uses the remaining capacity. An exchange of weight from lower to higher density cannot decrease value. If all suffix items fit, $\lambda = 0$ attains equality. If $R = 0$ and the suffix is nonempty, its largest density attains zero; an empty suffix is covered by $\lambda = 0$. Thus

$$U_d(R) = \min_{\lambda \in \{0\} \cup \{\rho_i : i > d\}} F_d(R, \lambda). \tag{5}$$

The identity includes tied densities, zero residual capacity, and all-fit suffixes.

3.1 Core preprocessing and interpretation

The oracle can be applied to an original instance or to a separately justified reduced core. In the core panel used below, a feasible greedy solution supplies an integer lower bound L. A variable is fixed only when the opposite assignment is infeasible or its LP bound is strictly below L. Every feasible solution with value at least L must then respect that fixing. Applying such individually necessary fixings together preserves all such solutions. Subtracting fixed-in weight and profit produces the residual instance; its threshold is shifted by the fixed-in profit. The bank is rebuilt for this residual instance and its own efficiency order. This argument does not preserve arbitrary low-threshold trees of the original instance. Exact reference optima are used only to check the transformations, not to select bank entries.

4 Certified multiplier banks

A bank B is a nonempty finite set of nonnegative rational numbers. Its bound is $U_d^B(R) = \min_{\lambda \in B} F_d(R, \lambda)$. Every such bank gives a valid upper bound, but this alone does not make its pruning decisions equal to those of the LP.

Lemma 1 (Root-tail sufficiency) *Let s be the first root index for which $\sum_{i\le s} w_i > W$. If it exists, the bank $\{0\} \cup \{\rho_i : i \ge s\}$ reproduces $U_d(R)$ at every feasible prefix. If every item fits at the root, the bank $\{0\}$ suffices.*

Proof If $d < s$, selecting an arbitrary prefix uses no more weight than selecting it entirely. Filling the suffix through any index $j < s$ therefore uses at most $\sum_{i\le j} w_i \le W$. A fractional split cannot occur before s. If $d \ge s$, all remaining items are already in the root tail. Equation (5) supplies a tail density, or zero for an all-fit suffix, attaining the LP value. When all root items fit, the remaining suffix fits after every prefix because omitted items only release capacity. Tied densities do not change the weight argument. □

4.1 Integer-threshold coverage

The set of feasible consumed weights after d decisions is

$$S_0 = \{0\}, \qquad S_{d+1} = \big(S_d \cup \{q + w_{d+1} : q \in S_d\}\big) \cap [0, W]. \tag{6}$$

Only states $(d, W - q)$ with $q \in S_d$ require coverage. Prefix profits need not be enumerated because they are integers.

Theorem 2 (Coverage criterion) *A bank preserves (3) at every feasible prefix and every integer threshold if and only if, for every reachable (d, R), it contains a multiplier satisfying*

$$F_d(R, \lambda) < \lfloor U_d(R) \rfloor + 1. \tag{7}$$

Proof Weak duality gives $U_d^B(R) \ge U_d(R)$. Hence (7) is equivalent to $\lfloor U_d^B(R) \rfloor = \lfloor U_d(R) \rfloor$. Adding the integer P preserves floor equality, which gives identical strict comparisons with every integer τ. Conversely, if the floors differ, take $\tau = P + \lfloor U_d(R) \rfloor + 1$. The LP rejects and the bank does not. This last necessity statement concerns all integer thresholds mathematically; the circuit contract only covers thresholds in its finite register. □

On a sorted density grid, convexity of $F_d(R, \lambda)$ makes the grid points satisfying (7) a nonempty contiguous index interval. The root-tail grid suffices by Lemma 1. Sort these intervals by right endpoint and select that endpoint whenever the current selection misses an interval. Each selected point covers its interval. The intervals that cause new selections are pairwise disjoint, so every bank on this grid needs at least as many points. This is a minimum-cardinality grid bank, with a checkable lower-bound certificate. The greedy stabbing method itself is standard [7].

4.2 Rational multipliers and open endpoints

For $j = d, \ldots, n$, put $V_{dj} = \sum_{d<i\le j} v_i$ and $W_{dj} = \sum_{d<i\le j} w_i$. Because positive reduced costs form a prefix,

$$F_d(R, \lambda) = \max_{j=d,\ldots,n} \{V_{dj} + \lambda(R - W_{dj})\}, \quad \lambda \ge 0. \tag{8}$$

Writing $K = \lfloor U_d(R) \rfloor + 1$, its continuous coverage interval is the intersection of

$$\lambda(R - W_{dj}) < K - V_{dj}, \quad j = d, \ldots, n, \qquad \lambda \ge 0. \tag{9}$$

Every finite endpoint is rational. The upper endpoint is open; a positive lower endpoint is open, while zero may be included. The upper endpoint may be infinite. Nonemptiness follows from an LP-optimal multiplier.

Proposition 3 (Minimum rational-bank certificate) *For the finite set of reachable states, a minimum-cardinality bank among nonnegative rational multipliers can be constructed by interval stabbing with these endpoint conventions.*

Proof Collect all finite endpoints, including zero. Choose a rational representative between each pair of consecutive endpoints and one above the greatest endpoint. Membership is

constant within each such cell. A point at zero can be moved a sufficiently small positive distance without losing membership in any of the finitely many intervals containing it. Any other included endpoint can likewise be moved slightly to its right: every finite upper endpoint is excluded. Thus cell representatives suffice. Process intervals by increasing upper endpoint. If the next interval is not already hit, select a representative immediately below its upper endpoint; for an unbounded interval use a rational point above the greatest finite endpoint. For a finite upper endpoint u, the selected representative lies below u and above every lower endpoint strictly below u. Every remaining interval hit by a point in the earliest-ending interval has lower endpoint below u and upper endpoint at least u, so this replacement preserves its coverage. This exchange establishes greedy optimality. Equivalently, the intervals triggering new points are pairwise disjoint, giving a lower bound equal to the number selected. □

This minimizes cardinality, not numerator size, denominator size, qubits, gates, or circuit depth. Choosing a midpoint or a mediant in a cell can change arithmetic widths without changing coverage.

For a concrete example, take $w = v = (1, 1)$ and $W = 1$. The singleton bank $B = \{1/2\}$ gives root bound $3/2$, whereas the LP value is 1. Nevertheless, it intersects the coverage intervals of all five reachable (d, R) states and preserves their integer-target decisions (Fig. 1). This analytical example is separate from the benchmark cohorts.

4.3 A sufficient bank without reachability enumeration

Let $\rho = a/b > 0$ be a reduced density and $G_\rho = \sum_{i:\rho_i=\rho} w_i$ the total weight of its tied group. Consider a state whose fractional split lies in that group. At ρ, the residual LP value has denominator dividing b, so its distance to the next integer is at least $1/b$. Between ρ and an adjacent distinct density, the slope of the dual objective has magnitude at most the weight of the remaining split group, and therefore at most G_ρ.

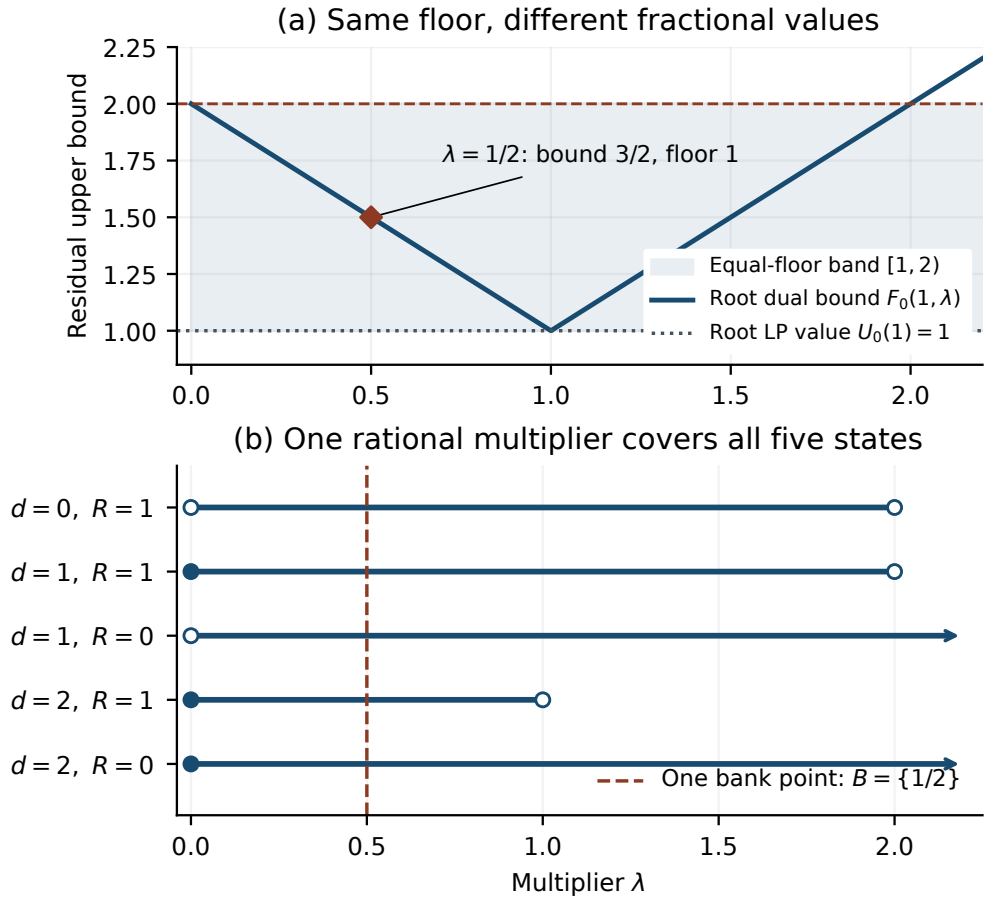


Fig. 1 Illustration of integer-threshold coverage. The root dual bound can differ from the LP value while retaining its floor. Below, one rational multiplier intersects all five state coverage intervals; open circles exclude endpoints, filled circles include them, and arrows denote unbounded intervals

Consequently,

$$|\lambda - \rho| < \frac{1}{bG_\rho} \quad \Longrightarrow \quad F_d(R, \lambda) < \lfloor U_d(R) \rfloor + 1, \tag{10}$$

provided that the segment crosses no other distinct density. To see the strict inequality, the increase above the optimum is at most $G_\rho|\lambda - \rho| < 1/b$, no larger than the available integer gap. Zero is retained for empty and all-fit suffixes, including zero-profit tails.

The implemented sufficient constructor tests each root-tail density and its immediate neighbors against (10), forms the resulting grid intervals, and stabs those intervals. This avoids the capacity-reachability dynamic program. Its minimality is only for

these sufficient constraints, not for the actual all-state coverage intervals. It can retain the entire root-tail grid. The full tied-group weight and strict inequality are essential; Appendix A gives failures of weaker formulations.

4.4 Preprocessing cost and a limitation

Reachability enumeration has up to $(n+1)(W+1)$ states. This is pseudopolynomial in numerical capacity and can be expensive compared with classical optimization itself. The grid compiler computes coverage intervals on those states. The rational compiler additionally intersects up to $n-d+1$ affine constraints per state. Preprocessing costs must therefore be included in any end-to-end performance assessment. If a compilation budget is exceeded, the constructors return the root-tail bank; its coverage is preserved, but minimality is not certified.

Proposition 4 (Linear cardinality can be necessary) *For each $m \geq 2$, there is an instance with $m+1$ items whose minimum rational bank has cardinality $m - \lfloor m/2 \rfloor + 1$.*

Proof For $i = 1, \ldots, m$, take $w_i = 2$, $v_i = 4(m-i+1)+1$. Append a dummy item with weight and profit 1. Set $k = \lfloor m/2 \rfloor$ and $W = 2k+1$. The last main density is $5/2$, so the dummy, of density 1, is last. For each $i = k+1, \ldots, m$, set the first $i-k-1$ decisions to zero. The greedy suffix fills k main items and half of item i. Its value is half-integral. Immediately below and above $\rho_i = v_i/2$, the dual slopes are -1 and 1. Adjacent distinct densities are at least $3/2$ away, so the exact coverage interval is $(\rho_i - 1/2, \rho_i + 1/2)$. Convexity prevents coverage from returning outside this interval. These $m-k$ intervals are pairwise disjoint, since their centers differ by 2. The all-zero full leaf adds $[0, 1/W)$, disjoint from all of them: $\rho_m = 5/2$ gives the lowest interval $(2, 3)$, whose lower endpoint $5/2 - 1/2 = 2$ exceeds $1/W$. Thus at least $m-k+1$ multipliers are needed. Conversely, zero and the main densities from the root split onward reproduce every suffix LP value: a main-item split uses a retained density, an all-fit suffix uses zero, and the unit-weight dummy cannot be partially filled with a strictly fractional amount when residual capacity is integer. These $m-k+1$ points attain the lower

bound. The dummy also makes the common weight divisor one. The lower bound concerns bank cardinality; the family itself is easy to optimize. □

5 Reversible implementation and comparison control

5.1 Constant-coefficient predicate

For a bank entry $\lambda = a/b$ with $a \geq 0$, $b > 0$, define

$$c_i = bv_i - aw_i, \qquad C_\lambda = aW + \sum_i \max(c_i, 0), \tag{11}$$

$$D_\lambda(x) = \sum_{i \leq d} \big[(1 - x_i)\max(c_i, 0) + x_i \max(-c_i, 0)\big]. \tag{12}$$

Direct expansion gives

$$b\{P + F_d(R, \lambda)\} = C_\lambda - D_\lambda(x), \qquad P + F_d(R, \lambda) < \tau \iff C_\lambda < D_\lambda(x) + b\tau. \tag{13}$$

The implementation compares the two nonnegative operands on the right. It need not represent a potentially negative difference on overweight inputs. All coefficients are compiled from the classical instance and bank. The circuit evaluates the decision predicate using integer coefficients, without quantum division or efficiency truncation.

Write $\mathrm{bitlength}(u) = \lceil \log_2(u + 1) \rceil$ for nonnegative integers u. Let $t = \max(1, \mathrm{bitlength}(1 + \sum_i v_i))$ and $\tau_{\max} = 2^t - 1$. For each entry,

$$0 \leq D_\lambda(x) + b\tau \leq M_\lambda := \sum_{i \leq d} |c_i| + b\tau_{\max}. \tag{14}$$

The shared accumulator has width

$$q = \max\Big\{1,\ 1 + \mathrm{bitlength}\Big(\max_{\lambda \in B}\{C_\lambda, M_\lambda\}\Big)\Big\}. \tag{15}$$

The weight word uses $1 + \mathrm{bitlength}(W + \sum_i w_i)$ bits, with a minimum of one. These conservative widths include padding for comparison. To form D_λ, initialize the sum of positive c_i over the prefix, subtract c_i controlled on x_i when $c_i > 0$, and add $-c_i$ when $c_i < 0$. Every intermediate remains nonnegative and is bounded by the sum of absolute prefix coefficients. Adding $b\tau$ is bounded by (14).

The Fourier-arithmetic implementation uses full, ideal quantum Fourier transform (QFT) additions and constant comparisons. Its arithmetic follows Fourier-basis addition [9] and the arithmetic primitives described by Wilkening et al., Supplement E.3–E.5 [28]. Controlled gates are decomposed into an explicit gate list [37]. No rotations are deliberately omitted. Numerical simulation uses floating-point angles; "exact" in the predicate claim refers to rational/integer semantics, not infinitely accurate physical rotations. A separate Boolean implementation uses X, controlled-NOT (CX), and Toffoli gates. The Boolean implementation requires strictly positive profits, whereas the QFT implementation accepts zero profits; their test domains are reported separately.

5.2 Output semantics and cleanup

At each fixed classical depth the desired transformation is

$$|x\rangle|\tau\rangle|z\rangle|0^a\rangle \longmapsto |x\rangle|\tau\rangle|z \oplus f(x,\tau)\rangle|0^a\rangle. \tag{16}$$

First compute the prefix weight and its capacity-rejection flag. For each multiplier, compute the loss accumulator, compare (13) into a fresh flag, and reverse the accumulator computation. Compute the logical OR of all bound flags and the capacity flag into an aggregate bit, copy it to the arbitrary output target, then reverse all preceding gates. Logical OR ensures that simultaneous rejection conditions still produce rejection. In the ideal arithmetic model, inverse computation restores every work bit and preserves the input registers. Theorem 2 supplies equivalence to the LP only when the supplied bank is certified. An arbitrary bank circuit correctly evaluating its own bound is not automatically an LP-equivalent circuit.

The resource comparison concerns fixed-depth bounding predicates. Integrating them into a quantum search requires coherent node handling and tree reflections, which are outside this implementation. An extension to superpositions of depths would also require depth-controlled prefix arithmetic; its synthesis costs are not included here.

Figure 2 separates instance-dependent classical preparation from reversible predicate evaluation.

5.3 Root-aware fractional-scan control

The control reconstructs a fractional greedy suffix scan using the same direct-control QFT engine, threshold width, output semantics, and inverse cancellation. It computes feasible whole-item takes, residual capacity, and the comparison at a possible fractional split. If s is the root split, no item $j < s$ can be the first nonfitting suffix item at a feasible prefix: used weight before j is at most $\sum_{i<j} w_i$. The control therefore omits fractional-product terms for $j < s$, while retaining its whole-item logic and separate capacity rejection. All-fit roots need only an integer comparison. Widths are recomputed for the eligible fractional positions and all encoded thresholds.

The root-aware control is reconstructed from the published arithmetic primitives, rather than taken from a released LP-oracle netlist [28, 29]. Both implementations use

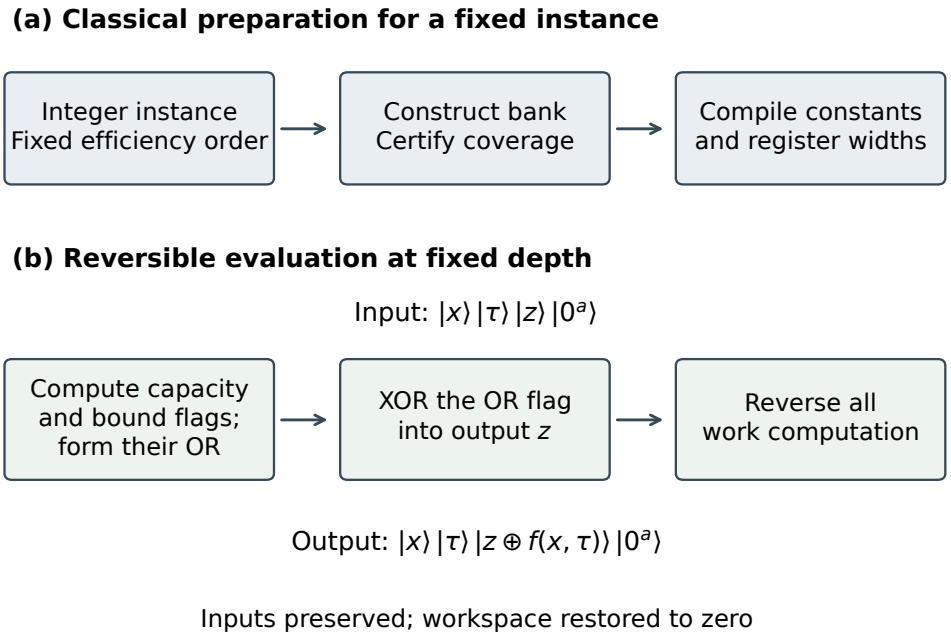


Fig. 2 Conceptual workflow, rather than a literal gate diagram. For a fixed instance and depth, the predicate f combines capacity rejection with the bank's bound flags. Copying their OR into the target and reversing the work preserves the inputs and restores clean workspace

serial shared workspace and the same local inverse-cancellation rule. The comparison is specific to these implementations and does not establish optimal circuit cost.

6 Evaluation protocol

6.1 Inputs and evidence categories

The main arithmetic panel contains 700 instances, and the core panel contains their 700 deterministically transformed residual instances. The original panel comprises five generator classes and seven sizes $n \in \{12, 14, 16, 18, 20, 22, 24\}$, with 20 trials per class–size pair. The seed is $1000n + j$ for trial $j = 0, \ldots, 19$, using NumPy's `RandomState`. Capacity is $\lfloor \sum_i w_i/2 \rfloor$. For uncorrelated inputs, weights and profits are drawn uniformly from integers 1–100. Weakly correlated inputs use such weights and

$v_i = \max(1, w_i + \eta_i)$, where η_i is uniform on integers $-10, \ldots, 10$. Strongly correlated inputs set $v_i = w_i + 10$; subset-sum inputs set $v_i = w_i$. Inverse-strongly-correlated inputs draw profits uniformly from integers 1–100 and set $w_i = v_i + 10$. Exact arrays and seeds are retained in the input registries. The 105-input resource cohort uses trials 0, 1, and 2 for every class–size pair, at depth $\lfloor n/2 \rfloor$, with the sufficient bank from (10). This selection is independent of the observed resource ratios. Appendix B specifies the registries and scripts.

Exact rational coverage and logical netlist counts are classified as DERIVED, propagation through gates as SIMULATED, and host elapsed times as MEASURED. Tolerances and computational budgets are fixed settings (HARDCODED). Logical gate counts and depth exclude physical routing, rotation synthesis, and device execution time.

6.2 Arithmetic coverage and independent references

For every original and core instance, a set-based reachability implementation enumerates all states in (6). It evaluates the full suffix density grid plus zero using exact rational/integer coefficients, independently of the production greedy-LP and bank constructors. The four tested banks are the root tail, minimum grid, sufficient bank without reachability enumeration, and continuous rational bank. Equality of residual floors covers all integer prefix profits and targets by Theorem 2; it is not an exhaustive quantum statevector experiment.

A separate primal reference enumerates LP vertices, including box vertices and vertices with one fractional coordinate. It checks 1,792 systematic two-item inputs with weights 1–4 and profits 0–3, including zero-capacity and all-fit transitions, plus 96 additional instances of size 3–7 using seed 202610251703. This reference avoids both the greedy routine and the dual-breakpoint reference. It checks interval endpoints and disjointness certificates separately. Counterexamples delimiting the assumptions are given in Appendix A.

6.3 Gate checks and resource accounting

Small-circuit tests use independently computed primal LP answers, both target-bit values, and initialized work registers. QFT tests use full rotations and numerical tolerance 10^{-8}. Boolean tests interpret the actual X/CX/Toffoli gate lists exactly. Fixed fixtures include near-integer fractions, ties, zero profits where supported, zero capacity, all-fit and empty inputs, maximum thresholds, and an intentionally insufficient bank. Table 4 records their numerical inputs. Coherent checks additionally compare complex amplitudes; basis truth tables alone would not detect unintended relative phases.

For resource accounting, Qiskit generates a $\{u, \mathrm{CX}\}$ decomposition template for each native gate type at optimization level zero. The profiler applies these templates to the circuit gate list, assuming all-to-all connectivity, and computes depth by dependency scheduling. The local profiler is cross-checked against complete transpilation on small controls. Active qubits include all interface wires plus every wire touched by a gate, even if an interface wire is otherwise idle. Allocated qubits are also retained. The common inverse-cancellation pass removes literal inverses separated only by gates acting on disjoint wires. The cohort uses budgets of 150,000 allocated qubits, 2,000,000 gates, and 60 seconds per build or profile operation. Capped and failed runs are retained with explicit status labels.

7 Results

7.1 All-state coverage

Table 2 reports exhaustive arithmetic coverage of the declared panels. All four banks agree with the independent residual-floor reference on 5,204,456 reachable states: 20,817,824 comparisons, with 0 mismatches. The 700 core cases retain all 23 all-fit instances, including 11 empty cores. All reachable states of these cases are included.

Table 2 Derived exact-arithmetic coverage. Each reachable state is compared with four banks; these are not quantum gate simulations

Panel	Inputs	States	Comparisons	Mismatches
Original	700	3,148,192	12,592,768	0
Reduced cores	700	2,056,264	8,225,056	0
Total	1,400	5,204,456	20,817,824	0

The 15 scaled and 5 tiny auxiliary records in the input registry are outside the declared 700-instance panel.

The separate primal-vertex audit covers 1,888 inputs and 15,999 feasible prefixes, with 0 root-tail value mismatches and 0 floor mismatches for each of the grid, sufficient, and rational banks. It checks 1,369 grid and 1,369 rational minimum-cardinality certificates, with 0 failures, and 1,637 independently reconstructed rational critical intervals, with 0 endpoint mismatches. These counts are different checks on overlapping inputs, not independent statistical samples.

7.2 Gate-level checks and initialization discrepancy

The completed QFT bank tests cover 24 method-cases and 1,120 basis columns, with 0 gate mismatches. Certified cases have 0 LP-predicate disagreements. The deliberately insufficient bank has 8 expected LP-predicate disagreements across the tested method rows, while its gate outputs agree with its supplied-bank predicate. The Boolean tests cover 7 cases and 2,216 gate columns, with 0 mismatches, including 548 invalid-depth encodings. Two primitive-unitary coherent checks span 64 logical amplitudes, with 0 failures at tolerance 10^{-8}. Each method-case is counted once; different circuit forms are not independent problem instances.

For $w = (2, 4, 1)$, $v = (1, 2, 0)$, $W = 3$, and $d = 1$, dense statevector simulations of the raw and inverse-cancelled 22-qubit oracle gave maximum complex-amplitude errors approximately 4.77×10^{-15} and 1.94×10^{-15}, respectively. Explicit primitive-gate or

Table 3 Derived paired resource ratios on 105 inputs. Lower/equal/higher compares the bank with the root-aware scan separately on each input

Resource	Median	Minimum	Maximum	Lower/equal/higher
Allocated qubits	0.4641	0.3974	0.5294	105/0/0
Active qubits	0.4908	0.3676	0.5455	105/0/0
CX gates	0.4981	0.0933	0.9180	105/0/0
Arbitrary-angle u gates	0.4995	0.0942	0.9178	105/0/0
Common-basis depth	1.0720	0.0798	2.0480	38/0/67

decomposed unitary preparation also passed on the matrix-product-state backend at tolerance 10^{-8}.[1]

The root-aware scan control was separately checked on 104 exact Boolean columns, 208 QFT basis columns across eight conditions, and one coherent condition spanning 16 logical amplitudes: 0 mismatches, dirty-workspace failures, or coherent failures. Four further complete-transpiler comparisons agree with the control profiler, with 0 discrepancies. These checks cover the stated small inputs; the 105-input resource cohort is not exhaustively gate-simulated.

7.3 Matched logical resources

All 105 root-aware resource comparisons completed, with 0 caps and 0 compilation errors. Table 3 gives per-instance bank/control ratios, summarized by their median, minimum, and maximum. Figure 3 shows every paired ratio. These quantities are logical resources derived from the compiled gate lists.

The median CX ratio is 0.4981 and the median active-qubit ratio is 0.4908. However, median depth is 1.0720 times the control depth: 67 inputs are deeper and 38 shallower, with a maximum depth ratio of 2.0480. The bank reuses one accumulator for successive compute–flag–uncompute evaluations, creating dependencies between

[1] A test using partial-register initialization on the same layout produced amplitude error approximately 0.2154. Preparation-only checks reproduced an incorrect state before any oracle gates. Its cause remains unresolved, and this preparation path is excluded from the supporting correctness evidence. The failed record is retained; the successful checks use the same oracle and tolerance.

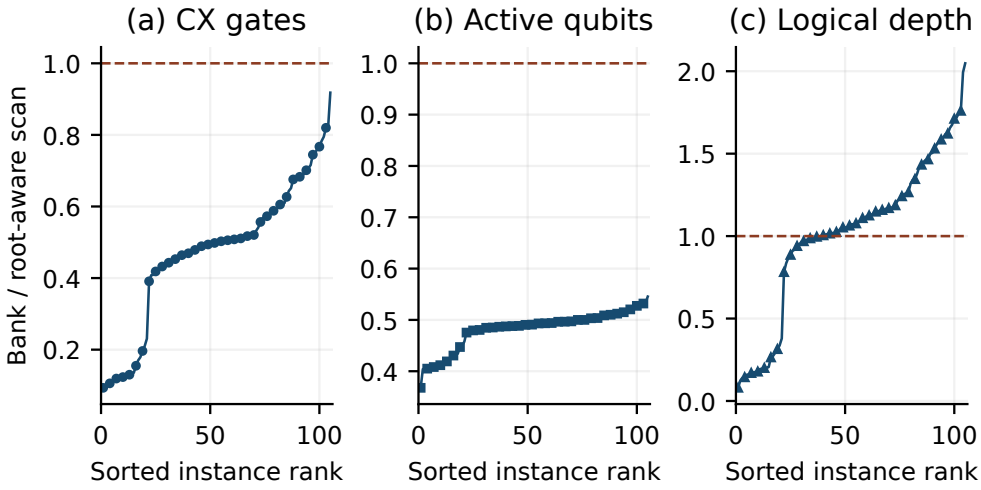


Fig. 3 Derived bank-to-root-aware-scan resource ratios for all 105 fixed-depth inputs. Each panel sorts its own ratios for readability, so horizontal positions are not matched instance identities across panels. A ratio below one favors the bank. The horizontal reference denotes equal resources

multipliers. The scan instead uses separate weight, profit, residual-capacity, and comparison registers in a single suffix pass. These layouts permit different degrees of parallelism, so fewer gates need not imply a shallower circuit. The median of paired ratios is not a ratio of aggregate totals and does not establish an average device-time saving.

The control omits fractional calculations before the earliest possible split, and active-qubit counts exclude unused workspace while retaining all interface wires. The reported ratios apply to this matched comparison, not to an optimal implementation or a classical knapsack solver.

8 Discussion and limitations

A certified bank preserves the LP rejection decision at integer targets while allowing its fractional value to differ. This permits a constant-coefficient circuit that avoids dynamically finding a fractional split. Coverage certificates are required because a merely valid Lagrangian bound can prune less than the LP. The root-tail construction is a safe baseline; smaller banks require additional coverage reasoning.

These results support a compiler for the stated prefix predicate and quantify its reversible resource tradeoff. Lagrangian multiplier sets and integer rounding already occur in prior work [6], and fractional-greedy bounds already occur in quantum knapsack B&B comparisons [28]. The contribution concerns the all-prefix coverage construction and its implementation; minimum bank cardinality does not imply minimum gate count.

The circuit contract requires positive integer weights, nonnegative integer profits, a fixed efficiency-ordered prefix, representable integer thresholds, and clean initial workspace. It returns no numerical LP value, integrality flag, completion witness, or LP-selected branching variable. Those outputs would require additional computation. Arithmetic coverage and gate-level simulation establish different properties; the initialization discrepancy demonstrates the need for preparation-only and independent dense-reference checks.

The resource comparison assumes ideal arbitrary-angle gates and all-to-all connectivity. Hardware costs would additionally depend on routing, rotation synthesis, and error management. A complete search implementation would still need coherent node handling, branching and marking operations, reflections, error allocation, witness recovery, and preprocessing. Quantum-query frameworks [4, 5] provide context, but do not supply the missing implementation costs. The reported oracle resource reductions therefore do not establish an end-to-end runtime advantage.

The resource tradeoff depends on the arithmetic layout and compilation model. Alternative implementations may change the relative gate counts, qubit requirements, and depth.

9 Conclusion

A certified static multiplier bank can preserve integer-threshold LP-pruning decisions for fixed efficiency-ordered knapsack prefixes. Coverage reduces to interval constraints,

and the predicate can be expressed through bounded constant-coefficient arithmetic with reversible cleanup. Exact arithmetic checks cover the original and reduced-core instances, and separate small-circuit simulations test the reversible predicate. The matched root-aware comparison finds lower CX counts and qubit requirements throughout its 105-input cohort, together with increased depth on most inputs. These findings support a specific oracle construction and an explicit resource tradeoff; they do not establish physical or end-to-end quantum advantage.

Statements and Declarations

Funding

Pending author confirmation before submission.

Competing interests

Pending declarations from all authors before submission.

Data and code availability

The input registries, circuit sources, numerical result ledgers, and verification scripts are retained in the research archive.

A Counterexamples delimiting the contract

These exact arithmetic examples falsify stronger statements, not the stated integer-prefix contract. With $w = (1, 1)$, $v = (1, 1)$, $W = 1$, and bank $\{1/2\}$, the root LP is 1 while the bank bound is $3/2$. Their floors agree, but the decisions differ at rational target $5/4$.

For nonprefix branching, take $w = (10, 9)$, $v = (100, 1)$, $W = 15$. The root-tail grid is $\{0, 1/9\}$. Fixing the lower-density item in first leaves the high-density item

with capacity 6: the total LP bound is 61, while the tail-bank bound is 905/9. They disagree at integer target 62. Prefix order is therefore a substantive assumption.

For tied groups, take $w = (2, 2, 3)$, $v = (1, 1, 1)$, $W = 1$. The LP is $1/2$, but $\lambda = 1/3$ gives bound 1. Its distance from the split density $1/2$ is $1/6$. A radius using one tied item's weight would admit it ($1/6 < 1/4$), whereas the required group-weight radius is $1/8$. For strictness, $w = v = (1)$ and $W = 0$ give LP 0; $\lambda = 0$ at distance 1 from density 1 gives bound 1, so the nonstrict radius fails at target 1.

Negative multipliers are unsafe: with $w = v = (1)$ and $W = 2$, $\lambda = -1$ gives 0 below the LP value 1. Integer weight normalization also need not preserve the fractional LP: $w = (2)$, $v = (3)$, $W = 1$ has LP $3/2$, whereas division of weight by 2 and flooring capacity gives LP 0. Equal integer feasible sets do not imply equal LP-pruning predicates.

B Gate fixtures and reproducibility map

Table 4 specifies the fixed QFT fixtures. Raw and inverse-cancelled forms and the recorded control policies produce method-cases; they are not independent problem samples. The Boolean suite omits the zero-profit fixture outside its validator domain and includes the empty instance and $w = (65537, 65539)$, $v = (3, 5)$, $W = 65538$. Its depth register includes invalid encodings. QFT simulation uses seed 2026102529 and full rotations; the retained metadata specifies each policy, encoded threshold range, and work-bit check.

The reproducibility records identify the input registries, circuit-test conditions, failed initialization test, source hashes, and numerical resource records. Tables and resource plots are generated from these records.

References

[1] Dantzig, G.B.: Discrete-variable extremum problems. Operations Research **5**(2),

Table 4 Fixed QFT bank fixtures; all listed inputs are hardcoded test settings. The last bank is deliberately not certified as LP-equivalent

Weights	Profits	W	d	Bank choice
$(17, 3)$	$(16, 2)$	1	1	Full density grid + zero
$(2, 4, 1)$	$(1, 2, 0)$	3	1	Full density grid + zero
$(2, 3)$	$(3, 1)$	10	2	$\{0\}$
$(3, 5)$	$(2, 4)$	0	1	Full density grid + zero
$(1, 1)$	$(1, 1)$	1	1	$\{1/2\}$
(2)	(1)	1	0	$\{0\}$, insufficient

266–288 (1957) https://doi.org/10.1287/opre.5.2.266

[2] Kellerer, H., Pferschy, U., Pisinger, D.: Knapsack Problems. Springer, Berlin, Heidelberg (2004). https://doi.org/10.1007/978-3-540-24777-7

[3] Balas, E., Zemel, E.: An algorithm for large zero-one knapsack problems. Operations Research **28**(5), 1130–1154 (1980) https://doi.org/10.1287/opre.28.5.1130

[4] Montanaro, A.: Quantum-walk speedup of backtracking algorithms. Theory of Computing **14**(15), 1–24 (2018) https://doi.org/10.4086/toc.2018.v014a015

[5] Montanaro, A.: Quantum speedup of branch-and-bound algorithms. Physical Review Research **2**(1), 013056 (2020) https://doi.org/10.1103/PhysRevResearch.2.013056

[6] Bożejko, W., Burduk, A., Pempera, J., Uchroński, M., Wodecki, M.: Optimal solving of a binary knapsack problem on a D-Wave quantum machine and its implementation in production systems. Annals of Operations Research **364**, 2111–2126 (2026) https://doi.org/10.1007/s10479-024-06025-1 . First published online 13 May 2024

[7] Erickson, J.: Algorithms. Self-published textbook, first edition (2019). https://

jeffe.cs.illinois.edu/teaching/algorithms/

[8] Vedral, V., Barenco, A., Ekert, A.: Quantum networks for elementary arithmetic operations. Physical Review A **54**(1), 147–153 (1996) https://doi.org/10.1103/PhysRevA.54.147

[9] Draper, T.G.: Addition on a Quantum Computer. arXiv:quant-ph/0008033 (2000). https://doi.org/10.48550/arXiv.quant-ph/0008033 . https://arxiv.org/abs/quant-ph/0008033

[10] Horowitz, E., Sahni, S.: Computing partitions with applications to the knapsack problem. Journal of the ACM **21**(2), 277–292 (1974) https://doi.org/10.1145/321812.321823

[11] Pisinger, D.: Core problems in knapsack algorithms. Operations Research **47**(4), 570–575 (1999) https://doi.org/10.1287/opre.47.4.570

[12] Pisinger, D.: A minimal algorithm for the 0-1 knapsack problem. Operations Research **45**(5), 758–767 (1997) https://doi.org/10.1287/opre.45.5.758

[13] Martello, S., Pisinger, D., Toth, P.: Dynamic programming and strong bounds for the 0-1 knapsack problem. Management Science **45**(3), 414–424 (1999) https://doi.org/10.1287/mnsc.45.3.414

[14] Beier, R., Vöcking, B.: Random knapsack in expected polynomial time. Journal of Computer and System Sciences **69**(3), 306–329 (2004) https://doi.org/10.1016/j.jcss.2004.04.004

[15] Dey, S.S., Dubey, Y., Molinaro, M.: Branch-and-Bound Solves Random Binary IPs in Polytime. Version 4, 16 September 2021; first submitted 2020 (2021). https://arxiv.org/abs/2007.15192v4

[16] Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC), pp. 212–219 (1996). https://doi.org/10.1145/237814.237866

[17] Brassard, G., Høyer, P., Mosca, M., Tapp, A.: Quantum amplitude amplification and estimation. In: Quantum Computation and Information. Contemporary Mathematics, vol. 305, pp. 53–74. American Mathematical Society, Providence, RI (2002). https://doi.org/10.1090/conm/305/05215 . https://arxiv.org/abs/quant-ph/0005055

[18] Bulger, D., Baritompa, W.P., Wood, G.R.: Implementing pure adaptive search with Grover's quantum algorithm. Journal of Optimization Theory and Applications **116**(3), 517–529 (2003) https://doi.org/10.1023/A:1023061218864

[19] Dürr, C., Høyer, P.: A quantum algorithm for finding the minimum. arXiv:quant-ph/9607014 (1996). https://arxiv.org/abs/quant-ph/9607014

[20] Bennett, C.H., Bernstein, E., Brassard, G., Vazirani, U.: Strengths and weaknesses of quantum computing. SIAM Journal on Computing **26**(5), 1510–1523 (1997) https://doi.org/10.1137/S0097539796300933

[21] Becker, A., Coron, J.-S., Joux, A.: Improved generic algorithms for hard knapsacks. In: Advances in Cryptology – EUROCRYPT 2011. Lecture Notes in Computer Science, vol. 6632, pp. 364–385. Springer, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-20465-4_21

[22] Bernstein, D.J., Jeffery, S., Lange, T., Meurer, A.: Quantum algorithms for the subset-sum problem. Cryptology ePrint Archive, Paper 2013/199. Expanded version of the PQCrypto 2013 paper (2013). https://eprint.iacr.org/2013/199

[23] Helm, A., May, A.: Subset sum quantumly in 1.17^n. In: 13th Conference

on the Theory of Quantum Computation, Communication and Cryptography (TQC 2018). Leibniz International Proceedings in Informatics, vol. 111. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2018). https://doi.org/10.4230/LIPIcs.TQC.2018.5 . Article 5, pp. 5:1–5:15

[24] Bonnetain, X., Bricout, R., Schrottenloher, A., Shen, Y.: Improved classical and quantum algorithms for subset-sum. In: Advances in Cryptology – ASIACRYPT 2020. Lecture Notes in Computer Science, vol. 12492, pp. 633–666. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-64834-3_22

[25] Jaques, S., Rattew, A.G.: QRAM: A survey and critique. Quantum **9**, 1922 (2025) https://doi.org/10.22331/q-2025-12-02-1922

[26] Hou, W., Perkowski, M.: Quantum-based algorithm and circuit design for bounded knapsack optimization problem. Quantum Information and Computation **20**(9–10), 766–786 (2020) https://doi.org/10.26421/QIC20.9-10-4

[27] Hamitouche, M., Hnaien, F., Menacer, D.E.: Towards a quantum resolution of the knapsack problem. In: Proceedings of the PRICAI 2025 Workshop on Quantum Computing for Search and Optimization Problems. CDMTCS Research Report Series. University of Auckland, Auckland, New Zealand (2025). Workshop paper. https://www.cs.auckland.ac.nz/research/groups/CDMTCS/researchreports/download.php?selected-id=904

[28] Wilkening, S., Lefterovici, A.-I., Binkowski, L., Perk, M., Fekete, S.P., Osborne, T.J.: A quantum algorithm for solving 0-1 knapsack problems. npj Quantum Information **11**, 146 (2025) https://doi.org/10.1038/s41534-025-01097-8

[29] Wilkening, S., Lefterovici, A.-I., Binkowski, L., Perk, M., Fekete, S.P., Osborne, T.J.: A quantum algorithm for solving 0-1 Knapsack problems. Zenodo. Software

and experiment data, version R1, released 18 August 2025 (2025). https://doi.org/10.5281/zenodo.16895828 . https://zenodo.org/records/16895828

[30] Christiansen, P.J., Binkowski, L., Ramacciotti, D., Wilkening, S.: Quantum tree generator improves QAOA state-of-the-art for the knapsack problem. In: 2025 IEEE International Conference on Quantum Computing and Engineering (QCE) (2025). https://doi.org/10.1109/QCE65121.2025.00011 . https://arxiv.org/abs/2411.00518v2

[31] Farhi, E., Goldstone, J., Gutmann, S.: A quantum approximate optimization algorithm. arXiv:1411.4028 (2014). https://arxiv.org/abs/1411.4028

[32] Gilliam, A., Woerner, S., Gonciulea, C.: Grover adaptive search for constrained polynomial binary optimization. Quantum **5**, 428 (2021) https://doi.org/10.22331/q-2021-04-08-428

[33] Sanavio, C., Tignone, E., Ercolessi, E.: Hybrid classical–quantum branch-and-bound algorithm for solving integer linear problems. Entropy **26**(4), 345 (2024) https://doi.org/10.3390/e26040345

[34] Gabbassov, E., Rosenberg, G., Scherer, A.: Lagrangian duality in quantum optimization: Overcoming QUBO limitations for constrained problems. Physical Review Research **7**, 023305 (2025) https://doi.org/10.1103/53gd-2374

[35] Martiel, S., Remaud, M.: Practical implementation of a quantum backtracking algorithm. In: SOFSEM 2020: Theory and Practice of Computer Science. Lecture Notes in Computer Science, vol. 12011, pp. 597–606. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-38919-2_49 . https://arxiv.org/abs/1908.11291

[36] Cuccaro, S.A., Draper, T.G., Kutin, S.A., Moulton, D.P.: A new quantum ripple-carry addition circuit. arXiv:quant-ph/0410184 (2004). https://doi.org/10.48550/arXiv.quant-ph/0410184 . https://arxiv.org/abs/quant-ph/0410184

[37] Barenco, A., Bennett, C.H., Cleve, R., DiVincenzo, D.P., Margolus, N., Shor, P., Sleator, T., Smolin, J.A., Weinfurter, H.: Elementary gates for quantum computation. Physical Review A **52**(5), 3457–3467 (1995) https://doi.org/10.1103/PhysRevA.52.3457

[38] Thapliyal, H., Varun, T.S.S., Munoz-Coreas, E.: Quantum Circuit Design of Integer Division Optimizing Ancillary Qubits and T-Count. arXiv:1609.01241 (2016). https://doi.org/10.48550/arXiv.1609.01241 . https://arxiv.org/abs/1609.01241

[39] Preskill, J.: Quantum computing in the NISQ era and beyond. Quantum **2**, 79 (2018) https://doi.org/10.22331/q-2018-08-06-79

[40] Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C.J., Lishman, J., Gacon, J., Martiel, S., Nation, P.D., Bishop, L.S., Cross, A.W., Johnson, B.R., Gambetta, J.M.: Quantum computing with Qiskit. arXiv:2405.08810 (2024). https://doi.org/10.48550/arXiv.2405.08810 . https://arxiv.org/abs/2405.08810